

BS21E 挂件端 Lux / UV 数据采集与传输协议 V2.0

1. 适用范围

本协议用于 BS21E 挂件端采集环境光照、紫外线、电池电量和时间信息，并完成：

- 本地 Flash 环形缓存；
- BLE 自动断点续传；
- APP 端实时展示；
- APP 端历史趋势统计；
- APP 端每日光环境总结；
- APP 端 UV 风险提示。

挂件端传感器为：

DFRobot SEN0540
核心芯片：LTR390-UV

2. 协议版本

协议名称：PENDANT_RECORD_V2
协议版本：2.0
单条记录长度：20 字节
字节序：小端序 Little Endian
采样周期：每 60 秒生成一条记录

V2 协议与旧版 16 字节协议不兼容。

升级固件后，建议清空旧版 Flash 环形缓存，避免 16 字节历史记录和 20 字节新记录混存。

3. 单条记录格式

字节偏移	长度	字段名称	数据类型	单位	说明
0-3	4 B	seq	uint32_t	无	数据序号，每生成一条新记录加 1
4-7	4 B	timestamp_s	uint32_t	秒	Unix 时间戳，使用 UTC 秒数
8-11	4 B	lux	uint32_t	lx	已换算后的环境光照强度
12-15	4 B	uv_raw	uint32_t	counts	UVS 通道原始计数值，不是标准 UVI
16	1 B	battery	uint8_t	%	电池百分比
17	1 B	flags	uint8_t	无	状态标志位

字节偏移	长度	字段名称	数据类型	单位	说明
18-19	2 B	crc	uint16_t	无	校验字段，当前版本暂时固定为 0x0000
20 B					

4. 字段说明

4.1 seq

```
uint32_t seq;
```

用途：

- 检查数据是否丢失；
- 支持断点续传；
- APP 端去重；
- 判断 Flash 缓冲区中的数据顺序。

规则：

每生成一条采样记录，seq 加 1。
即使手机未连接，也必须继续递增。
达到 uint32_t 最大值后允许自然回绕。

4.2 timestamp_s

```
uint32_t timestamp_s;
```

含义：

Unix 时间戳，单位为秒。
使用 UTC 时间，不直接存储北京时间。

规则：

- APP 与挂件完成 BLE 连接后，应向挂件端下发当前时间；
- 挂件端本地维护时间；
- APP 展示时自行转换为本地时区；
- 若尚未完成时间同步，则：

```
timestamp_s = 0  
flags.bit0 = 0
```

4.3 lux

```
uint32_t lux;
```

含义：

已换算后的环境光照强度，单位为 lx。

要求：

- 挂件端读取 ALS 原始寄存器；
- 在挂件端调用现有 `ltr390_get_lux()` 完成换算；
- 传输给 APP 的不是 ALS 原始计数值，而是换算后的 Lux；
- 浮点计算结果转换为整数时，采用四舍五入；
- 若读取失败：

```
lux = 0  
flags.bit1 = 0  
flags.bit5 = 1
```

4.4 uv_raw

```
uint32_t uv_raw;
```

含义：

UVS 通道原始 ADC 计数值。
不是标准 UV Index。

要求：

- 挂件端不直接换算 UVI；
- APP 端根据固定采集配置换算 UVI；
- 不允许将 `uv_raw` 直接展示为 UVI；
- 若读取失败：

```
uv_raw = 0  
flags.bit2 = 0  
flags.bit5 = 1
```

4.5 battery

```
uint8_t battery;
```

规则：

数值	含义
0-100	当前电池百分比
255	当前无法获得有效电量

4.6 flags

```
uint8_t flags;
```

统一约定：

1 = 状态成立
0 = 状态不成立

位定义：

bit	宏名称	1 表示
bit0	FLAG_TIME_VALID	时间戳有效，已完成时间同步
bit1	FLAG_LUX_VALID	本条记录中的 Lux 数据有效
bit2	FLAG_UV_VALID	本条记录中的 UV 原始数据有效
bit3	FLAG_LUX_SATURATED	Lux 数据发生饱和或截断
bit4	FLAG_UV_SATURATED	UV 原始数据发生饱和或截断
bit5	FLAG_SENSOR_ERROR	本次采集过程中出现传感器异常
bit6	RESERVED	预留，当前写 0
bit7	RESERVED	预留，当前写 0

正常采集且时间有效时：

```
flags = 0x07
```

即：

```
bit0 = 1  
bit1 = 1  
bit2 = 1
```

4.7 **crc**

```
uint16_t crc;
```

当前版本暂未启用 CRC 校验，统一写入：

```
crc = 0x0000
```

APP 端在 V2.0 中不执行 CRC 校验。

后续若启用 CRC，应升级协议版本，不应在不修改版本号的情况下静默改变字段含义。

5. LTR390 采集配置

LTR390 的 ALS 和 UVS 共用测量通道。每次采样需要切换模式。

5.1 ALS 配置

```
模式：ALS  
增益：3×  
分辨率：18 bit  
输出：换算后的 lux
```

5.2 UVS 配置

```
模式：UVS  
增益：18×  
分辨率：20 bit  
输出：uv_raw 原始计数值
```

采用该 UVS 配置后，APP 端可以按照：

```
UVI = uv_raw / 2300.0
```

换算标准 UV Index。

注意：

2300 counts / UVI 只适用于 18× 增益 + 20 bit 分辨率。

如果未来修改 UVS 增益或分辨率，必须同步升级协议版本，或者增加设备配置字段。APP 不应猜测挂件端当前使用的采集配置。

6. 每分钟采样流程

每 60 秒执行一次完整采样。

推荐流程：

1. 配置 ALS：
 gain = 3×
 resolution = 18 bit
2. 切换至 ALS 模式。
3. 等待 data_ready 置位。
4. 读取 ALS 原始数据。
5. 调用 ltr390_get_lux() 完成 Lux 换算。
6. 配置 UVS：
 gain = 18×
 resolution = 20 bit
7. 切换至 UVS 模式。
8. 等待 data_ready 置位。
9. 读取 UVS 原始计数值 uv_raw。
10. 读取电池电量。
11. 生成 seq 和 timestamp_s。
12. 设置 flags。
13. 组装 20 字节记录。
14. 写入 Flash 环形缓存。
15. 若 BLE 已连接，则向 APP 发送。

要求：

- ALS 和 UVS 模式切换后，必须等待新的 `data_ready`；
- 不允许直接读取上一次测量遗留的旧数据；
- 若采用“丢弃一次旧值”的策略，也必须确保最终写入 Flash 的数据来自新一轮有效测量。

7. BLE 传输要求

7.1 单条发送

推荐每次 BLE Notification 发送一条完整记录：

单条记录：20 字节

BLE 默认 ATT MTU 为 23 字节时，Notification 可承载 20 字节有效数据，因此单条 V2 记录可以完整发送。

如果现有 BLE 数据包还包含额外头部，需要：

- 协商更大的 MTU；或
- 将头部放入独立控制特征值；或
- 采用拆包方案。

优先建议：

数据特征值直接发送 20 字节记录，不额外添加头部。

7.2 断点续传

APP 与挂件连接后：

1. APP 告知挂件端最近已成功接收的 seq。
2. 挂件端从下一个 seq 开始补发。
3. 按 seq 升序发送历史记录。
4. 历史记录补发完成后，再继续发送实时记录。

APP 端处理原则：

相同 seq 的记录只保存一次。
发现 seq 缺口时，可以请求重新补发。

8. Flash 环形缓存

Flash 中每条记录使用与 BLE 完全一致的 20 字节格式。

每分钟采样一次时：

```
20 B × 60 × 24  
= 28800 B / 天  
≈ 28.1 KB / 天
```

建议至少保留：

```
1 天完整数据
```

环形缓存写满后：

```
覆盖最旧记录。
```

升级至 V2 协议后，旧版 16 字节 Flash 缓存应清空，或者单独实现版本迁移逻辑。

比赛阶段建议直接清空旧缓存，减少兼容成本。

9. C 结构体参考

```
#include <stdint.h>

#pragma pack(push, 1)

typedef struct {
    uint32_t seq;
    uint32_t timestamp_s;
    uint32_t lux;
    uint32_t uv_raw;
    uint8_t battery;
    uint8_t flags;
    uint16_t crc;
} pendant_record_v2_t;

#pragma pack(pop)

_Static_assert(sizeof(pendant_record_v2_t) == 20,
    "pendant_record_v2_t must be exactly 20 bytes");
```

建议发送 BLE 和写入 Flash 时使用显式序列化，确保所有多字节字段均按照小端序写入。

10. 示例数据

假设：

```
seq      = 31
timestamp_s = 0x67C5618F
lux      = 331 lx
uv_raw   = 41 counts
battery  = 86%
flags    = 0x07
crc      = 0x0000
```

则 20 字节原始数据为：

```
1F 00 00 00
8F 61 C5 67
4B 01 00 00
29 00 00 00
56
07
00 00
```

合并后：

```
1F 00 00 00 8F 61 C5 67 4B 01 00 00 29 00 00 00 56 07 00 00
```

解析结果：

字段	值
seq	31
timestamp_s	1740988815
lux	331 lx
uv_raw	41 counts
battery	86%
flags	0x07
crc	0x0000

11. 从 V1 升级至 V2 的差异

对比项	V1	V2
单条记录长度	16 B	20 B
lux 类型	uint16_t	uint32_t
uv_raw 类型	uint16_t	uint32_t
battery	uint8_t	uint8_t
flags	含义未完全冻结	明确位定义
crc	预留	仍预留，固定为 0x0000
每天 Flash 占用	约 22.5 KB	约 28.1 KB

V2 的主要目的：

避免户外强光环境下 Lux 被截断；
避免 UVS 高增益、高分辨率采集时 uv_raw 溢出；
统一挂件端、APP 端和云端的数据语义。

12. 挂件端完成后需要回复的确认项

请确认以下事项：

1. 是否已经将单条记录升级为 20 字节？
2. lux 是否使用 uint32_t，并且存储的是换算后的 lx？
3. uv_raw 是否使用 uint32_t，并且存储的是 UVS 原始计数值？
4. ALS 是否固定为 3× 增益 + 18 bit 分辨率？
5. UVS 是否固定为 18× 增益 + 20 bit 分辨率？
6. ALS ↔ UVS 切换后，是否等待 data_ready 再读取？
7. flags 是否按照本协议定义？
8. Flash 环形缓存是否已经按照 20 字节记录重新初始化？
9. BLE Notification 是否可以完整发送单条 20 字节记录？
10. APP 端是否需要同步修改断点续传解析逻辑？